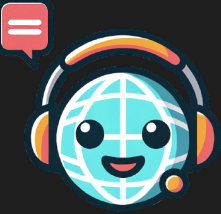


# POLYGLOT WORKSHOP

## *SETTING UP*

**Tutor**  
*Aniket Sen*

# WHY WE NEED TO SET UP?



- ▶ This is a hands-on workshop → we'll be coding a bunch of stuff
- ▶ Working with code → version control nightmare

## WHAT I MEAN?

```
1 import torch
2
3 scaler = torch.cuda.amp.GradScaler()
```

works fine for `PyTorch >= 1.6`

but for `PyTorch >= 2.4` we get

```
ModuleAttributeError: module 'torch.cuda.amp' has no attribute 'GradScaler'
```

## WHY?

because in `PyTorch 2.4`, `GradScaler` was relocated to `torch.amp.GradScaler`

# WHY WE NEED TO SET UP?



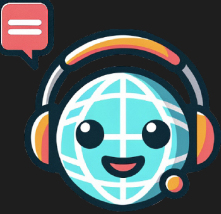
- ▶ We will also be working together → so we need to be on the same page
- ▶ So that all our bugs bug us equally 😊

## HOW CAN WE ALL BE ON THE SAME PAGE?

- ▶ We all work on the same environment
- ▶ We all work with the same software stack

**IT ALSO MAKES LIFE MUCH EASIER FOR US!!** 😊💧

# HOW CAN WE BE ON THE SAME ENVIRONMENT?



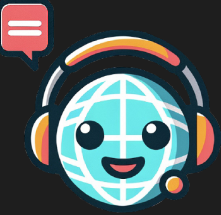
- ▶ We use *Marvin*!!

*Marvin* is a State of the Art (for 2022) Tier 3 HPC cluster at the University of Bonn

- ▶ We have created temporary accounts on Marvin for all of you
  - ▶ you should have received an email (please say now if you haven't)
- ▶ We have also reserved a few nodes for the workshop
- ▶ We will be doing everything on Marvin remotely

**A HUGE THANKS TO THE HPC TEAM OF UNI BONN!**

# HOW DO WE WORK REMOTELY ON MARVIN?



- Using **SSH** (Secure Shell)
- For that we need to set up a SSH key pair for you to access Marvin securely

Open **Terminal** on your laptop and type the following command

```
ssh-keygen -t ed25519
```

you will be asked to enter a file name to save the key

```
Generating public/private ed25519 key pair.  
Enter file in which to save the key (./home/<username>/.ssh/id_ed25519):
```

you will also be asked to enter a passphrase

```
Enter passphrase for "/home/<username>/.ssh/id_ed25519" (.empty for no passphrase):
```

**IMPORTANT!! Please enter a strong passphrase and remember it, otherwise you won't be able to access Marvin**

# PROVIDING THE PUBLIC KEY TO MARVIN



- Go to the Marvin user portal: <https://freeipa.hpc.uni-bonn.de/>
- Log in with the credentials you received via email from HPC@UniBonn

Account Settings

|                               |                                                                      |
|-------------------------------|----------------------------------------------------------------------|
| User login                    | demouser_hpc                                                         |
| Password                      | *****                                                                |
| Password expiration           | 2049-04-03 08:36:59Z                                                 |
| UID                           | 5161                                                                 |
| GID                           | 5161                                                                 |
| Principal alias               | demouser_hpc@HPC.UNI-BONN.DE                                         |
| Kerberos principal expiration |                                                                      |
| Login shell                   | /bin/bash                                                            |
| Home directory                | /home/demouser_hpc                                                   |
| SSH public keys               | <div>Add</div>                                                       |
| Certificates                  | <div>Add</div>                                                       |
| Certificate mapping data      |                                                                      |
| User authentication types     | <input type="checkbox"/> Password<br><input type="checkbox"/> RADIUS |

- Click on **SSH public keys** -> **Add** on the right side of the page
- Copy the content of your public key file (e.g., `~/.ssh/id_ed25519.pub`) and paste it into the **Public key** field

Set SSH key

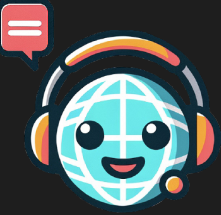
SSH public key:

```
ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AAAAIK0wmlN/Cr3JXqmLW7u+g9pTh+wvyqDhpSQEIQczKkVx9q demouser_hpc@demo_pc
```

Set Cancel

- Click on **Set** to save the public key

# PROVIDING THE PUBLIC KEY TO MARVIN



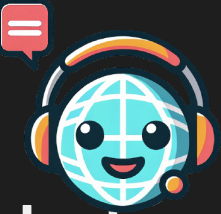
- Go to the Marvin user portal: <https://freeipa.hpc.uni-bonn.de/>
- Log in with the credentials you received via email from HPC@UniBonn

The screenshot shows the 'Entity Settings' page in the Marvin user portal. The top navigation bar includes tabs for 'Settings', 'User Groups (1)', 'Netgroups', 'Roles', 'HBAC Rules', 'Sudo Rules', and 'Subordinate ids'. Below the navigation bar, there is a toolbar with buttons for 'Refresh', 'Revert', 'Save' (highlighted with a red box), and 'Actions'. The main content area contains several input fields for user information: 'Job Title', 'First name' (with a red asterisk), 'Last name' (with a red asterisk), 'Full name' (with a red asterisk), 'Display name', 'Initials', 'GECOS', and 'Class'. The 'First name' field contains 'Demo', 'Last name' contains 'User', 'Full name' contains 'Demo User', 'Display name' contains 'Demo User', 'Initials' contains 'DU', and 'GECOS' contains 'Demo User'.

And finally, click **Save** in the top left bar to save the changes to your account

**If something went wrong, please tell us now!**

# SET UP AN SSH CONFIG FILE



- ▶ To make it easier to connect to Marvin, we will set up an SSH config file on your laptop
- ▶ This will allow us to connect to Marvin using a simple command like `ssh marvin` instead of `ssh -i /path/to/your/private/key <username>@marvin.hpc.uni-bonn.de`

Create a file named `config` in the `~/.ssh/` directory (if it doesn't already exist)

And add the following content to the `config` file

```
Host marvin
  HostName marvin.hpc.uni-bonn.de
  User <username>
  IdentityFile /path/to/your/private/key (.e.g., ~/.ssh/id_ed25519)
  IdentitiesOnly yes
```



# LET'S TEST IT OUT!



Open a new terminal and type the following command

```
ssh marvin
```

If everything is set up correctly, you will be greeted with this

```
Welcome to the marvin-cluster!
For more information check our wiki: https://wiki.hpc.uni-bonn.de

Do not hesitate to contact marvin-support@hpc.uni-bonn.de if you
have any questions or issues.

*** IMPORTANT ***

There is an issue with the lustre-filesystem. Some users are writing
absurd amount of small files to the MetaDataTargets, blocking them for
everyone else. Please make sure to understand lustre and use it
properly.
The issues regarding the home filesystem are resolved.
Please note the changes regarding ssh-login after our
downtime on 2024-09-25. Please refer to our wiki for further information
here https://wiki.hpc.uni-bonn.de/gaining_access#connecting-via-ssh
```

```
Last login: Fri Mar  6 10:25:11 2026 from [REDACTED]
```

```
Your Lustre Quota Usage:
```

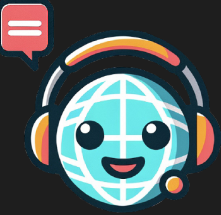
```
Disk quotas for usr asen_hpc (uid 225605):
```

| Filesystem | used | quota | limit | grace | files | quota | limit | grace |
|------------|------|-------|-------|-------|-------|-------|-------|-------|
| /lustre    | 452k | 0k    | 0k    | -     | 17    | 0     | 0     | -     |

```
[asen_hpc@login02 ~]$
```

Let us know now if you do not see this welcome message after running `ssh marvin`

# LET'S CREATE A WORKSPACE FOR YOU!



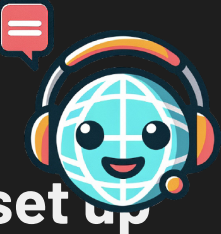
In the terminal where you are connected to Marvin, run the following command to create a workspace for you

```
wget -O ~/create_workspace_polyglot.sh https://tinyurl.com/polyglot-st-cr \  
&& chmod +x ~/create_workspace_polyglot.sh \  
&& ~/create_workspace_polyglot.sh
```

This will create a workspace for you in the `/lustre/scratch/data/<username>-polyglot` directory

Next we will set up some environment variables that will be useful for us later on

# SETTING UP ENVIRONMENT VARIABLES



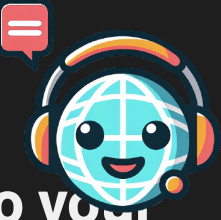
In the terminal where you are connected to Marvin, run the following command to set up the environment variables

```
wget -O ~/set_env_vars_polyglot.sh https://tinyurl.com/polyglot-st-env \
&& source ~/set_env_vars_polyglot.sh
```

This will set up the following environment variables for you:

- ▶ **WS\_DIR** : the directory of your workspace (e.g., `/lustre/scratch/data/<username>-polyglot` )
- ▶ **cdws** : a shortcut command to change to your workspace directory (i.e., `cd $WS_DIR` )
- ▶ **WS\_COMMON** : the directory of the common files that we will be using in the workshop (`/lustre/scratch/data/nklugeco_hpc-workshop` )
- ▶ **sbatch\_poly** : a shortcut command to submit a job to the cluster using the correct reservation and account (`sbatch --reservation=polyglot_[mon,tue...] --account=tmp_polyglot` )

# SETTING UP ENVIRONMENT VARIABLES



To make the environment variables available throughout the workshop, we add it to your `.bashrc` (this is a script that runs every time you log in to Marvin)

```
echo "source ~/set_env_vars_polyglot.sh" >> ~/.bashrc
```

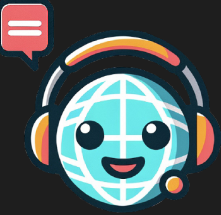
**Its completely understandable if you don't want to do this (especially if you already use Marvin regularly)**

**In that case, you need to run the following every time you start a new session on Marvin**

```
source ~/set_env_vars_polyglot.sh
```

**That was a lot of setup (and working with a terminal can be intimidating), so let's take a break!!**

# LET'S MAKE THINGS SLIGHTLY EASIER FOR YOU!



Working with the terminal can be intimidating, especially if you are not used to it

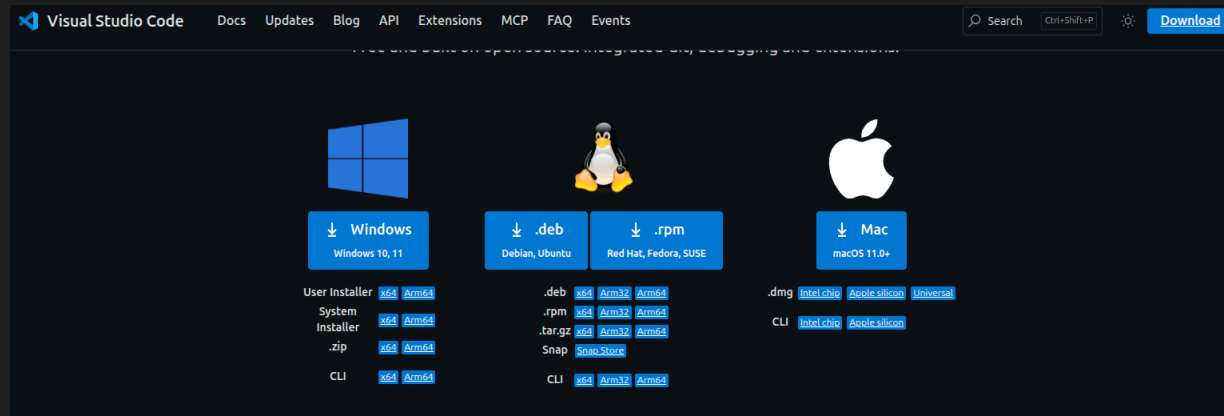
To make things easier for you, we would do everything through **VSCode**

**This is completely optional, you can still follow along using the terminal if you prefer**

You can download **VSCode** from the official website:

<https://code.visualstudio.com/download>

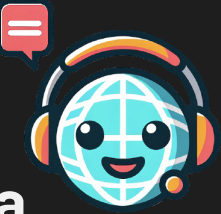
Choose the option that suits your system



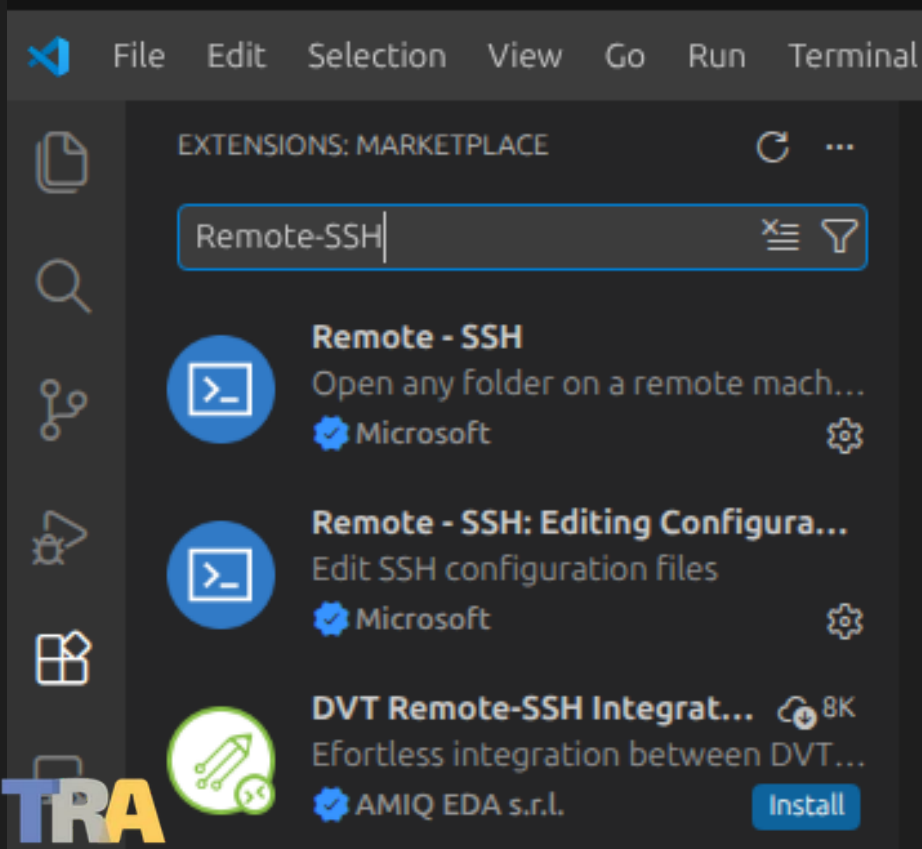
**TRA** free to ask us for help if you are not sure which one to choose or if you have any issues installing **VSCode**

**POLYGLOT 13**

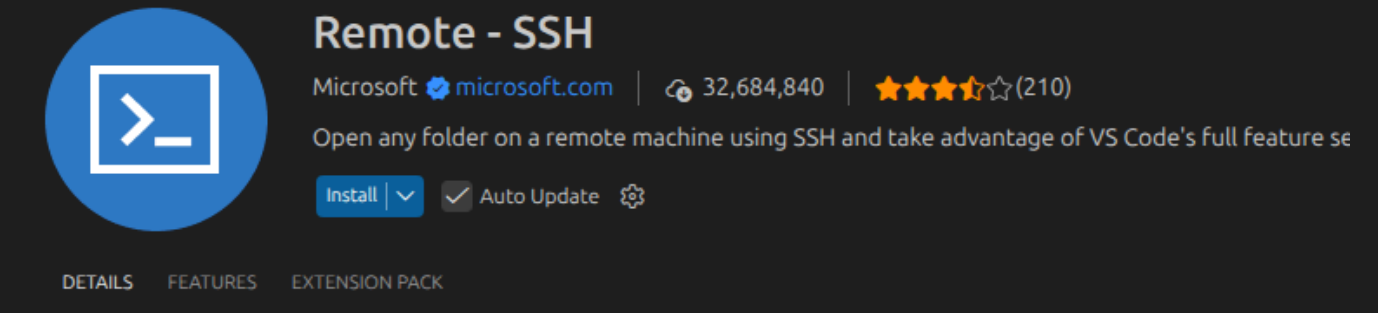
# REMOTE DEVELOPMENT WITH VSCODE



**VSCode** has a great extension called **Remote - SSH** that allows you to connect to a remote server (like Marvin) and work with the files on that server as if they were on your local machine

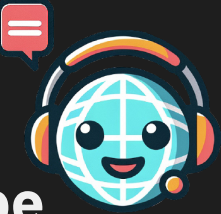


Got to the extensions tab in **VSCode** and search for **Remote - SSH** and install it

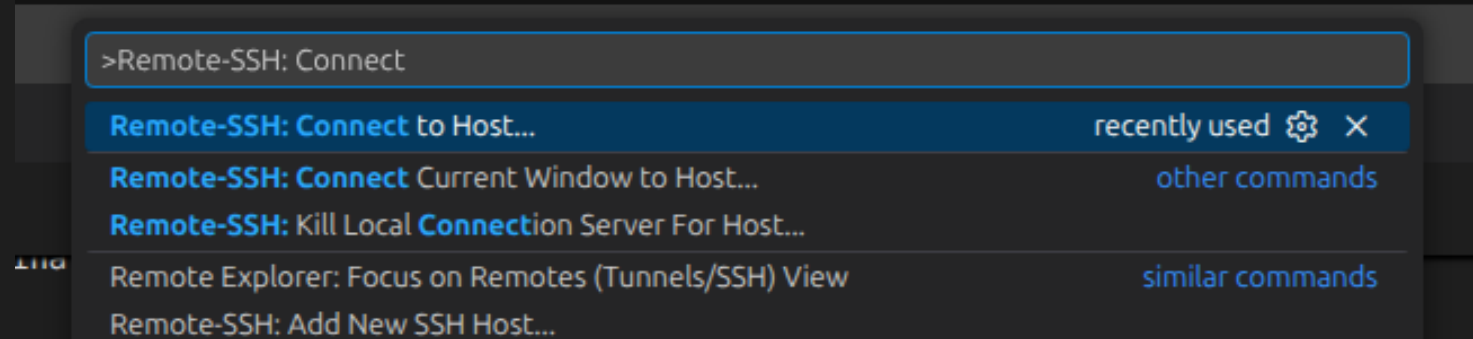


Visual Studio Code Remote - SSH

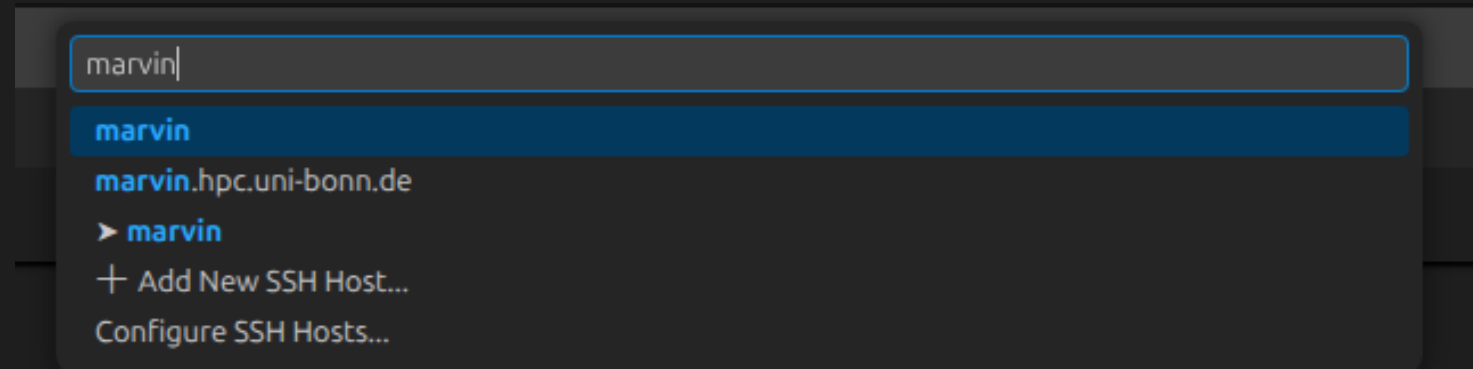
# CONNECTING TO MARVIN WITH VSCODE



On **VSCode**, press **F1** or **Ctrl + Shift + P** to open the command palette and type



Select **marvin** from the list of hosts (if you have many)

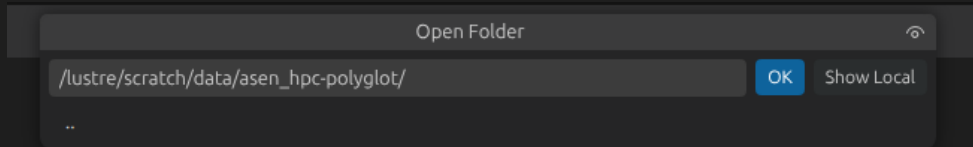


# SETTING UP VS CODE FOR REMOTE DEVELOPMENT



Now that we are connected to Marvin with **VSCode**, we open the workspace directory that we created earlier

Press **Ctrl + k** followed by **Ctrl + o** (or **File -> Open Folder**) and navigate to your workspace directory



We can also open up a terminal in **VSCode** at this directory by going to **Terminal -> New Terminal** (or pressing **Ctrl + Shift + `**)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

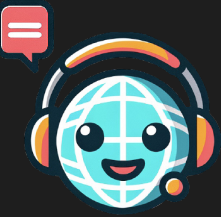
Your Lustre Quota Usage:
Disk quotas for user asen_hpc (uid 225605):
  Filesystem    used    quota   limit   grace   files   quota   limit   grace
    /lustre     456k     0k      0k      -        18        0        0      -
[asen_hpc@login01 asen_hpc-polyglot]$
```

**PERFECT!! NOW WE ARE ALL WORKING WITH THE SAME ENVIRONMENT** 🎉

This is going to be our main window for the rest of the workshop



# NOW WE NEED TO SET UP THE SOFTWARE STACK



Now that we are all working on the same environment, we need to set up the same software stack for everyone

We will ensure this by setting up an identical **Python** virtual environment for everyone

Run the following command in the terminal (either in **VSCode** or the regular terminal) connected to Marvin

```
wget -O ~/create_venv_polyglot.sh https://tinyurl.com/polyglot-st-venv \  
&& chmod +x ~/create_venv_polyglot.sh \  
&& ~/create_venv_polyglot.sh
```

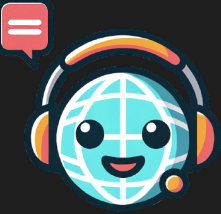
**THIS IS GOING TO TAKE A WHILE... GOOD TIME TO GET SOME COFFEE!**

Just to be certain, let's run the command again

**GREAT! NOW WE HAVE THE SAME SOFTWARE STACK SET UP FOR EVERYONE**



# LET'S RUN A SIMPLE PROGRAM



GEMM(General Matrix Multiply)s are at the heart of deep learning [[Pete Warden](#)]

So we will run a simple program that multiplies two square matrices together using

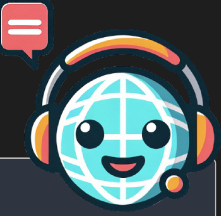
PyTorch

Let's get the exercise with the following command

```
cdws
git clone https://github.com/Polyglot/ex_setup.git
cd ex_setup
```

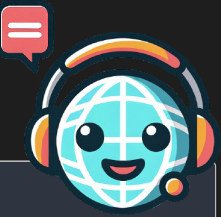
## LET'S TAKE A LOOK AT THE CODE!

# TORCH\_MM.PY



```
1 import torch
2 import time
3 import argparse
4
5 N_warm = 100
6 N_bench = 1000
7
8 def benchmark_mm(N : int):
9     A = torch.rand(N, N)
10    B = torch.rand(N, N)
11    # warmup
12    for _ in range(N_warm):
13        C = torch.mm(A, B)
14    start_time = time.time()
```

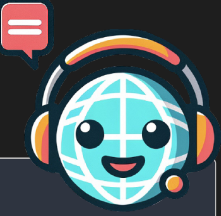
# TORCH\_MM.PY



```
1 import torch
2 import time
3 import argparse
4
5 N_warm = 100
6 N_bench = 1000
7
8 def benchmark_mm(N : int):
9     A = torch.rand(N, N)
10    B = torch.rand(N, N)
11    # warmup
12    for _ in range(N_warm):
13        C = torch.mm(A, B)
14    start_time = time.time()
```

**import** the necessary libraries ( **torch** , **time** , and **argparse** )

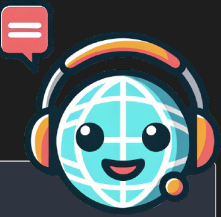
# TORCH\_MM.PY



```
1 import torch
2 import time
3 import argparse
4
5 N_warm = 100
6 N_bench = 1000
7
8 def benchmark_mm(N : int):
9     A = torch.rand(N, N)
10    B = torch.rand(N, N)
11    # warmup
12    for _ in range(N_warm):
13        C = torch.mm(A, B)
14    start_time = time.time()
```

Define the number of warmup iterations ( `N_warm` ) and the number of benchmark iterations ( `N_bench` ) for the matrix multiplication

# TORCH\_MM.PY

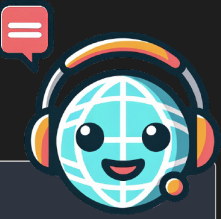


```
1 import torch
2 import time
3 import argparse
4
5 N_warm = 100
6 N_bench = 1000
7
8 def benchmark_mm(N : int):
```

Define the `benchmark_mm` function which benchmarks the `torch.mm` function for multiplying two square matrices of size `N x N`

- ▶ It creates two random matrices `A` and `B` of size `N x N`
- ▶ It performs `N_warm` warmup iterations of matrix multiplication
- ▶ It then performs `N_bench` benchmark iterations of matrix multiplication and times it with `time.time()`

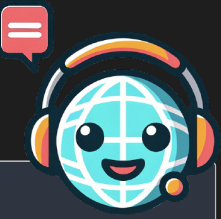
# TORCH\_MM.PY



```
1 import torch
2 import time
3 import argparse
4
5 N_warm = 100
6 N_bench = 1000
7
8 def benchmark_mm(N : int):
9     A = torch.rand(N, N)
10    B = torch.rand(N, N)
11    # warmup
12    for _ in range(N_warm):
13        C = torch.mm(A, B)
14    start_time = time.time()
```

The `main` function parses the command line arguments to get the size of the matrices `N` and calls the `benchmark_mm` function with that size

# TORCH\_MM.SH

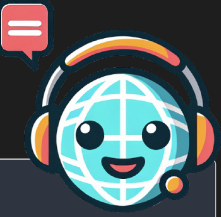


```
1 #!/bin/bash
2 #SBATCH --ntasks=1
3 #SBATCH --cpus-per-task=1
4 #SBATCH --partition=mlgpu_devel
5 #SBATCH --time=00:10:00
6 #SBATCH --mem=16G
7 #SBATCH --oversubscribe
8
9 source ~/set_env_vars_polyglot.sh
10 source ${MOD_FILE}
11 source ${WSDIR}/.venv_polyglot_t1/bin/activate
12 mkdir -p ${WSDIR}/ex_setup/logs
13 out_file="${WSDIR}/ex_setup/logs/log.torch_mm.${SLURM_JOB_ID}.out"
14 err_file="${WSDIR}/ex_setup/logs/log.torch_mm.${SLURM_JOB_ID}.err"
```

This is a **SLURM** job script that we will use to run the **torch\_mm.py** program on the cluster



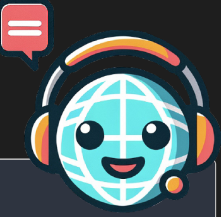
# TORCH\_MM.SH



```
1 #!/bin/bash
2 #SBATCH --ntasks=1
3 #SBATCH --cpus-per-task=1
4 #SBATCH --partition=mlgpu_devel
5 #SBATCH --time=00:10:00
6 #SBATCH --mem=16G
7 #SBATCH --oversubscribe
8
9 source ~/set_env_vars_polyglot.sh
10 source ${MOD_FILE}
11 source ${WSDIR}/venv_polyglot_t1/bin/activate
```

- It requests resources for 1 task
- with 1 cpu per task
- on the `mlgpu_devel` partition
- for a maximum time of 10 minutes

# TORCH\_MM.SH



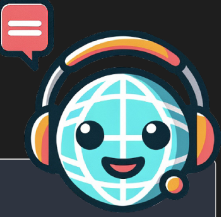
```
1 #!/bin/bash
2 #SBATCH --ntasks=1
3 #SBATCH --cpus-per-task=1
4 #SBATCH --partition=mlgpu_devel
5 #SBATCH --time=00:10:00
6 #SBATCH --mem=16G
7 #SBATCH --oversubscribe
8
9 source ~/set_env_vars_polyglot.sh
10 source ${MOD_FILE}
11 source ${WSDIR}/venv_polyglot_t1/bin/activate
```

- Set the environment variables
- Load the necessary modules
- Activate the **Python** virtual environment that we set up earlier
- Create a directory for the logs if it doesn't already exist

► Define the output and error log file paths using the **SLURM\_JOB\_ID** environment variable

**POLYGLOT 26**

# TORCH\_MM.SH



```
1 #!/bin/bash
2 #SBATCH --ntasks=1
3 #SBATCH --cpus-per-task=1
4 #SBATCH --partition=mlgpu_devel
5 #SBATCH --time=00:10:00
6 #SBATCH --mem=16G
7 #SBATCH --oversubscribe
8
9 source ~/set_env_vars_polyglot.sh
10 source ${MOD_FILE}
11 source ${WSDIR}/.venv_polyglot_t1/bin/activate
12 mkdir -p ${WSDIR}/ex_setup/logs
13 out_file="${WSDIR}/ex_setup/logs/log.torch_mm.${SLURM_JOB_ID}.out"
14 err_file="${WSDIR}/ex_setup/logs/log.torch_mm.${SLURM_JOB_ID}.err"
```

- Define the size of the matrices **N** (you can change this to benchmark different sizes)
- Run the **torch\_mm.py** program using **srun**

# LET'S RUN THE CODE!



you can run the `torch_mm.sh` script using the following command

```
sbatch_poly torch_mm.sh
```

You can check the status of your job using the following command

```
squeue -u <username>
```

Once the job is finished, take a look at the output log file to see the results

```
cat ${WSDIR}/ex_setup/logs/log.torch_mm.<SLURM_JOB_ID>.out
```

What time did you get for the matrix multiplication? Try changing the size of the matrices `N` and see how it affects the time taken

# NOW SOME HANDS-ON EXERCISES



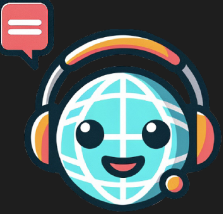
**1. LET'S TRY TO RUN THE CODE WITH MORE THREADS**

**2. LET'S TRY TO RUN THE CODE ON A GPU**

**3. WHAT HAPPENS IF WE WORK WITH LESSER PRECISION?**

**FEEL FREE TO PLAY AROUND WITH THE CODE AND SEE HOW IT AFFECTS THE PERFORMANCE OF THE MATRIX MULTIPLICATION**

# THAT'S ALL, FOLKS!



## QUESTIONS?